Revised Warm-Up Video Document and Technical Appendix

Multi-Agent Systems, SearchClient implementation, benchmark evidence, and reproducible testing instructions

Prepared from the current workspace on 2026-04-26. The purpose of this document is to directly answer the lecturer feedback: provide a better structure, explain the algorithmic ideas in enough detail to evaluate them, and include the concrete code and test procedure.

Submission contents:

- This PDF report, which explains the solver, its design choices, its limitations, and how to reproduce the results.
- The full code in `searchclient_python/` together with the original level set and `server.jar` in the submission bundle.
- Stored benchmark data in `searchclient_python/benchmark_results_2026-04-19.json` and fresh live-run artifacts in `submission_artifacts/logs/`.
- Replayable server logs for recording an improved video with the official GUI.

Direct response to the lecturer feedback. The document below is structured as: problem definition, solver architecture, detailed algorithm description, code mapping, experimental evidence, how to test, and a suggested video script.

1. Problem and Deliverables

The warm-up assignment asks for a client that solves hospital/MAPF-style levels using search. A level contains walls, colored agents, colored boxes, and goal cells. A solution is a sequence of joint actions that moves every box to its matching letter goal and every agent to its agent goal if such a goal is present.

The deliverable in this revised submission is not only a video script. It is a technical package: the code itself, reproducible commands, benchmark data, replay logs, and this written explanation that connects the algorithmic ideas to concrete source files.

Main files to inspect.

Component	Location	Purpose
CLI entry point	searchclient_python/main.py:5-14	Declares the available search strategies and max-memory option.
Level parser and planner selection	searchclient_python/searchclient/client.py:19-143	Parses colors, walls, boxes, goals, and chooses BFS/DFS/A*/WA*/gre
Phase trigger and phase planner	searchclient_python/searchclient/client.py:218-299	Enables phased decomposition on large multi-agent instances and ru
Generic graph search	searchclient_python/searchclient/graphsearch.py:15-116	Maintains frontier, explored set, best-g values, and goal termination.
Best-first frontier	searchclient_python/searchclient/frontier.py:84-125	Priority queue with deterministic tie-breaking and stale-entry suppress
Successor generation and pruning	searchclient_python/searchclient/state.py:167-339	Generates successors, ranks active agents, and filters actions toward
Applicability, result, and conflicts	searchclient_python/searchclient/state.py:343-482	Implements Move/Push/Pull legality, state transition, and multi-agent c
Heuristic computation	searchclient_python/searchclient/heuristic.py:17-269	Precomputes wall-aware distances and defines A*, WA*, and greedy e
Benchmark artifact	searchclient_python/benchmark_results_2026-04-19.json	Stored benchmark results used for the comparison tables in this repor

2. Solver Architecture

At a high level, the client follows a simple pipeline:

```
read level from server stdin
  -> build initial State(walls, boxes, agents, goals, colors)
  -> choose planning mode
       small / simple instances: one global search
       large multi-agent instances: phased decomposition + tail search
  -> output one joint action per time step to the server
```

- The CLI supports `-bfs`, `-dfs`, `-astar`, `-wastar [w]`, and `-greedy` (`main.py:5-14`).
- The initial state is parsed in `client.py:19-113`; walls, boxes, goals, and colors are stored in class variables of `State`.
- Search is executed through a generic graph-search loop in `graphsearch.py:15-116` and different frontier implementations in `frontier.py`.
- Multi-agent branching is controlled mainly inside `state.py`, where the code decides which agents may act and which actions are worth expanding.

3. State Representation and Legality Tests

The state representation is explicit. A state stores three dynamic objects: the row of each agent, the column of each agent, and a 2D array of box letters. The parent state, the joint action used to reach the state, and the path cost g are also stored. Walls, goals, and colors are static class-level data (`state.py:22-35`).

The action set contains `NoOp`, four `Move` actions, twelve `Push` actions, and eight `Pull` actions (`action.py:13-72`). Applicability is checked per action type in `state.py:343-383`:

- **Move:** the destination cell must be inside the grid, not a wall, not occupied by a box, and not occupied by another agent.
- **Push:** the adjacent cell in the agent movement direction must contain a box of the same color as the agent, and the target box destination must be free.
- **Pull:** the agent destination must be free, and the source box must be located behind the agent relative to the box movement direction; the pulled box must have the same color as the agent.

When a joint action is applied, `state.py:387-423` copies the state and updates agents and boxes deterministically. For the full synchronous model, conflict tests are implemented in `state.py:427-482`, covering agent-agent collisions, edge swaps, two agents grabbing the same box, two boxes moving to the same cell, and agent-box destination conflicts.

4. Baseline Graph Search

The graph-search loop is textbook in structure but includes one important practical improvement: a `best_g` map is maintained so that stale frontier entries can be ignored when a cheaper path to the same state has already been generated (`graphsearch.py:72-115`). That matters for best-first strategies implemented on a heap.

```
frontier.add(initial_state)
best_g[initial_state] = 0
explored = set()
while frontier not empty:
    state = frontier.pop()
    if state.g != best_g[state]:
        continue  # stale heap entry
    if goal_test(state):
        return extract_plan(state)
    explored.add(state)
    for neighbor in expand(state):
        if neighbor not in explored and (neighbor not in best_g or neighbor.g < best_g[neighbor]):
            best_g[neighbor] = neighbor.g
            frontier.add(neighbor)
```

- **BFS** uses a FIFO queue and is the clean uninformed baseline.
- **DFS** uses a LIFO stack and is included mainly to satisfy the assignment requirements and compare search behavior.

- **A***, **WA***, and **Greedy** all use the best-first frontier in `frontier.py:84-125` with different evaluation functions defined in `heuristic.py:244-269`.

5. Heuristic Design

The heuristic implementation begins from the assignment idea of counting unsatisfied goals, but the active configuration in the code is a stronger distance-based heuristic (`heuristic.py:32-35` sets `HEURISTIC_MODE = "goal_dist"`).

Why the heuristic is stronger than simple Manhattan distance.

- Before search starts, the code runs a BFS from each goal cell over the static wall grid (`heuristic.py:49-53` and `214-235`).
- These precomputed maps return the shortest wall-respecting path length from any reachable cell to the goal. This is more informative than Manhattan distance because walls are respected.
- For agent goals, the heuristic sums the precomputed distance from each agent to its own goal (`heuristic.py:126-135`).
- For box goals, the heuristic groups boxes by letter, then greedily matches each goal cell to the nearest currently available box of the same letter (`heuristic.py:137-174`).
- An extra term encourages an agent of the correct color to approach an unsolved box it can manipulate (`heuristic.py:175-200`).
- If static deadlock pruning is enabled and a state contains deadlocks, a large penalty is returned (`heuristic.py:117-123`).

Important honesty note: because the heuristic uses greedy box-goal assignment and an added agent-to-box encouragement term, it should be treated as an aggressive guiding heuristic rather than a formally admissible lower bound. Therefore the mode exposed as `-astar` is best understood as `f(n) = g(n) + h(n)` best-first search with a strong heuristic, not as a guarantee of optimality.

The implementation of the evaluation functions is direct: A* uses `g + h`, weighted A* uses `g + w*h`, and greedy uses `h` only (`heuristic.py:244-269`).

6. Multi-Agent Branching Reduction

The main challenge in the multi-agent levels is branching. If each agent can choose several actions, the naive joint-action successor function grows as the Cartesian product of those action sets. The code therefore defaults to a reduced successor generator in `state.py:248-339`.

Design choice 1: sequential joint-action mode.

The class variable `State.JOINT_ACTION_MODE` is set to `"sequential"`. In this mode, a successor usually contains one non-`NoOp` action and all other agents perform `NoOp` (`state.py:284-315`). This is a deliberate approximation: it drastically cuts branching, but it also means the searched plan space is more asynchronous than the full synchronous server model.

Design choice 2: active-agent selection.

- The code collects all unsolved boxes grouped by color (`state.py:169-182`).
- Agents are ranked by Manhattan distance to the nearest unsolved box of their color (`state.py:190-201`).
- At most two agents are kept active by default (`State.MAX_ACTIVE_AGENTS = 2`), and a rotating fallback agent is added so that one agent is not starved forever (`state.py:289-299`).

Design choice 3: action filtering.

- For each active agent, applicable actions are split into box-moving actions and pure moves (`state.py:212-246`).
- If the agent has useful unsolved boxes of its color, moves are ranked by whether they reduce the distance to the nearest target.
- If box actions are available, they are preferred, optionally with one improving move added.

- If only move actions are available, only the best one or two moves are expanded. This is controlled by `State.MAX_MOVE_ACTIONS_PER_AGENT = 2`.

These pruning ideas are pragmatic rather than theoretically clean. They work because the warm-up levels are small enough that reducing branching helps far more than broadening the search helps, but they can miss solutions or yield longer plans on tightly coupled instances.

7. Phased Decomposition in Detail

The lecturer specifically commented that the previous explanation of phased decomposition was not detailed enough. This section therefore states the exact trigger, ordering rule, inner objective, search strategy, stopping criterion, and fallback mechanism as implemented in `client.py:218-299`.

Trigger condition.

- Phased planning is never used for BFS or DFS (`client.py:219-221`).
- For informed search, it is enabled only when the level has at least three agents and at least ten unsolved boxes in the initial state (`client.py:222`).
- This means the decomposition is reserved for the larger instances where a flat global search became too expensive.

Agent ordering.

Agents are sorted by the distance from the agent to the nearest unsolved same-color box (`client.py:232-235`). Intuitively, the algorithm starts with the agent that is already closest to useful work.

Per-color letter list.

For a chosen agent, the code enumerates the unsatisfied goal letters whose color matches that agent's color (`client.py:183-195` and `242-244`). Example: if an agent is blue and the unsatisfied blue box goals are D, E, F, G, then the phase sequence for that agent is exactly D, E, F, G in the order they are discovered while scanning the goal grid.

Phase search itself.

```
for agent in sorted_agents:
    ACTIVE_AGENT_IDS = {agent}
    for letter in unsatisfied_letters_for_color(agent_color):
        phase_root = clone(current_state)
        phase_frontier = WA*(w = 15)
        phase_goal = all goals with this letter are satisfied
        phase_plan = search(phase_root, phase_frontier, phase_goal, max_expanded = 40000)
        if phase_plan exists:
            execute phase_plan into current_state and append it to the global prefix
ACTIVE_AGENT_IDS = None
if current_state is not yet goal:
    run one final global search from the new current_state
```

- The per-phase search always uses weighted A* with weight 15, regardless of the outer strategy (`client.py:248-252`). This is an aggressive speed choice.
- The phase goal is not the full level goal. It is only that all goals for one letter become satisfied (`client.py:253-257` together with `175-180`).
- The phase search is capped at `max_expanded = 40,000` nodes (`client.py:264-269`). If the phase does not finish, the code logs that fact and moves on to the next letter.
- If the phased prefix already solves the entire level, the algorithm stops. Otherwise, a final tail search is launched from the resulting intermediate state using the strategy originally requested on the command line (`client.py:285-299`).
- If no complete solution is found but some useful prefix has been generated, the current code returns the best-effort prefix so the GUI still shows progress (`client.py:295-299`).

Why this works in practice. The decomposition turns one hard global coordination problem into a sequence of smaller subproblems that are much easier for weighted A* to solve. The price is that the decomposition can be myopic: it commits to a color and letter order that may later be inconvenient.

Live trace of the phased planner on `MAthomasAppartment.lvl`.

The workspace contains a fresh run log in `submission_artifacts/logs/MAthomasAppartment_astar_2026-04-26.log`. Summing the final status line of each phase gives 5,589 expanded states and 28,411 generated states before the final 633-action solution was reported. This is more informative than the stored benchmark JSON, which only captured the last phase's status line for phased runs.

Phase	Agent	Letter	Expanded	Generated	Time s
Blue:D	1 (Blue)	D	106	497	0.263
Blue:E	1 (Blue)	E	0	0	0.272
Blue:F	1 (Blue)	F	0	0	0.281
Blue:G	1 (Blue)	G	0	0	0.291
Cyan:H	2 (Cyan)	H	5,006	25,946	10.373
Cyan:I	2 (Cyan)	I	87	402	10.594
Cyan:J	2 (Cyan)	J	0	0	10.614
Cyan:K	2 (Cyan)	K	0	0	10.628
Cyan:L	2 (Cyan)	L	52	235	10.846
Purple:M	3 (Purple)	M	42	172	10.922
Purple:N	3 (Purple)	N	0	0	10.931
Red:C	0 (Red)	C	215	869	11.284
Red:B	0 (Red)	B	0	0	11.315
Red:A	0 (Red)	A	81	290	11.608

8. Experimental Evidence

Two kinds of evidence are included: a stored benchmark table from 2026-04-19 and fresh live runs executed while preparing this document on 2026-04-26. The stored benchmark file is useful for broad comparison across algorithms. The fresh runs confirm that the current workspace still reproduces representative results.

Stored benchmark excerpt.

The table below comes from `searchclient_python/benchmark_results_2026-04-19.json`. For non-phased runs it is reliable directly. For phased runs such as `MAthomasAppartment`, the reported expanded/generated counts reflect only the final phase status line, not the cumulative total; that caveat is handled explicitly in the live-run section.

Level	Strategy	Solved	Expanded	Generated	Search s	Plan len
SAsimple0	bfs	Yes	18	24	0.013	5
SAsimple0	astar	Yes	13	23	0.011	5
SAsimple0	wastar5	Yes	5	13	0.009	5
SAsimple0	greedy	Yes	5	13	0.009	5
SAsimple2	bfs	Yes	1,359	1,625	0.128	30
SAsimple2	astar	Yes	36	54	0.014	30
SAsimple2	wastar5	Yes	48	66	0.015	46
SAsimple2	greedy	Yes	48	66	0.016	46
SAsimple3	bfs	No	-	-	-	-
SAsimple3	astar	No	-	-	-	-
SAsimple3	wastar5	No	-	-	-	-
SAsimple3	greedy	No	-	-	-	-
MAExample	bfs	Yes	2,318	2,478	0.393	38
MAExample	astar	Yes	824	972	0.178	38
MAExample	wastar5	Yes	58	122	0.022	42
MAExample	greedy	Yes	54	119	0.020	42
MAsimple1	bfs	Yes	80,724	95,004	23.307	26
MAsimple1	astar	Yes	3,349	6,290	1.318	26
MAsimple1	wastar5	Yes	27	135	0.023	26
MAsimple1	greedy	Yes	27	135	0.022	26
MAthomasAppartment	bfs	No	215,999	449,278	119.503	-
MAthomasAppartment	astar	Yes	81	290	6.320	633
MAthomasAppartment	wastar5	Yes	81	290	6.311	633
MAthomasAppartment	greedy	Yes	81	290	6.397	633

Key observations from the stored benchmark data.

- On `SAsimple2`, BFS expands 1,359 states, whereas the current g+h best-first mode expands only 36 and still returns a 30-step plan.
- On `MAsimple1`, BFS expands 80,724 states and takes 23.307 s; greedy and WA*(5) solve the same level in 27 expansions and roughly 0.02 s, with the same reported plan length 26.
- On `MAExample`, A* reduces generated states from 2,478 to 972 relative to BFS, while greedy/WA*(5) reduce it further to around 120 at the cost of a slightly longer plan.
- Some instances remain unsolved, for example `SAsimple3` and `MAsimple3`. The report therefore does not claim universal robustness.
- The large `MAthomasAppartment` instance is exactly where phased decomposition matters: BFS times out after generating 449,278 states in the stored run, whereas the informed phased planner completes the instance.

Fresh live verification runs (2026-04-26).

Live run artifact	Solved	Expanded	Generated	Actions	Wall s	Notes
MAsimple1_greedy	Yes	27	135	26	0.040	single search
MAthomasAppartment_astar	Yes	81	290	633	11.684	phased
SAsimple2_astar	Yes	36	54	30	0.021	single search

The fresh artifacts also include replay logs produced by the official server: `submission_artifacts/logs/SAsimple2_astar_replay_2026-04-26.log`, `submission_artifacts/logs/MAsimple1_greedy_replay_2026-04-26.log`, and `submission_artifacts/logs/MAthomasAppartment_astar_replay_2026-04-26.log`.

9. How to Run and Test the Code

The goal of this section is that a lecturer or TA can test the solver immediately. The required runtime is minimal: Python 3 and Java 11+ are enough. The only Python dependency declared in `pyproject.toml` is `psutil == 6.1.1`, used only for memory reporting.

Basic commands.

```
cd searchclient_python
python3 main.py -h

# Single-agent example
java -jar ../server.jar -l ../levels/SAsimple2.lvl \
  -c "python3 main.py -astar --max-memory 4096" -t 60 -s 0

# Multi-agent example
java -jar ../server.jar -l ../levels/MAsimple1.lvl \
  -c "python3 main.py -greedy --max-memory 4096" -t 60 -s 0

# Larger phased example
java -jar ../server.jar -l ../levels/MAthomasAppartment.lvl \
  -c "python3 main.py -astar --max-memory 4096" -t 60 -s 0
```

How to generate an official replay log for the video.

```
cd searchclient_python
java -jar ../server.jar -l ../levels/MAthomasAppartment.lvl \
  -c "python3 main.py -astar --max-memory 4096" \
  -t 60 -o ../submission_artifacts/logs/my_run.log

cd ..
java -jar server.jar -r submission_artifacts/logs/my_run.log -g -p -s 200
```

The replay command is the recommended way to record the revised video, because it uses the official course server GUI with pause, step, playback speed, and fullscreen support. The server help confirms these flags (`-g`, `-p`, `-s`, `-o`, and `-r`).

10. Suggested Structure for the Revised Video

The lecturer asked for a better-structured video. A simple way to improve the resubmission is to follow an explicit script and tie every spoken claim to visible code or visible execution output. The following outline is a concrete template for a 9-10 minute recording.

- **0:00-0:45:** State the assignment, show the repository structure, and say what is included in the submission package.
- **0:45-2:00:** Explain the state representation: walls, agents, boxes, goals, colors, and why color constraints matter.
- **2:00-3:15:** Show the generic graph-search loop and the frontier implementations. Make it clear how BFS/DFS differ from best-first search.
- **3:15-5:00:** Explain the heuristic carefully: precomputed wall-aware distances, box-goal matching, and the trade-off between speed and admissibility.

- **5:00-7:15:** Explain multi-agent branching reduction and then the phased decomposition algorithm step by step, including the exact trigger condition, WA*(15), per-letter subgoals, and fallback tail search.
- **7:15-8:15:** Show benchmark results and compare at least one single-agent level and one multi-agent level.
- **8:15-9:15:** Replay the official server log on a representative level and narrate what the agents are doing.
- **9:15-10:00:** End with limitations: the heuristic is aggressive, pruning sacrifices guarantees, and some levels remain unsolved.

The important improvement over the previous submission is that the video should not only show that the system works. It should also explain why the code is written this way, what each approximation buys, and what it gives up.

11. Limitations and Honest Assessment

- The distance heuristic is tuned for performance, not optimality guarantees. The `-astar` mode is therefore not formally optimal A*.
- Sequential joint-action mode and action pruning reduce branching dramatically, but they can also miss globally coordinated solutions or produce longer plans than a richer synchronous search would.
- Static deadlock pruning is disabled by default because classical Sokoban deadlock rules are not generally sound in a domain with Pull actions (`state.py:18-20`).
- The benchmark JSON should be read carefully on phased runs because a naive parser records only the final phase's status line. This report corrects that by also including the fresh phase-by-phase live trace.
- Some benchmark levels remain unsolved, notably `SAsimple3` and `MAsimple3`, so the solver is best presented as a strong warm-up implementation rather than a complete MAPF planner.

12. Conclusion

The revised submission now contains what the lecturer asked for: a structured explanation of the algorithmic ideas, a detailed account of the phased decomposition strategy, concrete mappings from those ideas to code files, reproducible commands, benchmark data, and replayable logs for a better video. The strongest technical idea in the implementation is not any single search strategy by itself; it is the combination of (1) wall-aware heuristic guidance, (2) branching reduction in multi-agent successor generation, and (3) phased subgoal planning on the larger instances.

If this document is followed as a recording script, the revised video should be far easier to evaluate than the earlier attempt because every major design decision is now explicit and testable.